

# Package ‘dsClinicalDataHarmonization’

September 15, 2026

**Title** dsClinicalDataHarmonization server side base functions

**Version** 0.0.1

**Date** 2026-09-14

**Author** Arianna Fontanesi, Simone Avesani, Gospel Ozioma Nnadi

**Maintainer** Arianna Fontanesi <arianna.fontanesi@univr.it>

**Description** dsClinicalDataHarmonization -  
Federated MDRRA Clinical Data Harmonization and data cleaning on DataSHIELD.

**AggregateMethods** cast\_NADS,  
cast\_to\_factorDS,  
check\_categoricalDS,  
check\_missing\_dataDS,  
check\_numericDS,  
check\_patient\_visitDS,  
check\_residual\_naDS,  
check\_variablesDS,  
clinical\_variable\_dictionaryDS,  
compute\_derived\_variablesDS,  
detect\_type\_mismatchesDS,  
detect\_zero\_anomaliesDS,  
drop\_rows\_missingDS,  
dummy\_encode\_ordinalDS,  
exploratory\_analysisDS,  
fill\_missing\_dataDS,  
force\_convert\_typesDS,  
global\_imputationDS,  
harmonization\_diagnosisDS,  
harmonization\_summaryDS,  
longitudinal\_imputationDS,  
post\_imputation\_cleanupDS,  
remove\_columnsDS,

**Options** datashield.privacyLevel=5

**Depends** R (>= 3.5)

**Encoding** UTF-8

**LazyData** true

**License** GPL-3

**SystemRequirements** GNU Fortran

**Config/roxygen2/version** 8.1.0

**RoxygenNote** 7.3.2

## R topics documented:

|                                          |           |
|------------------------------------------|-----------|
| cast_NADS . . . . .                      | 2         |
| cast_to_factorDS . . . . .               | 3         |
| check_categoricalDS . . . . .            | 3         |
| check_missing_dataDS . . . . .           | 4         |
| check_numericDS . . . . .                | 4         |
| check_patient_visitDS . . . . .          | 5         |
| check_residual_naDS . . . . .            | 5         |
| check_variablesDS . . . . .              | 6         |
| clinical_variable_dictionaryDS . . . . . | 7         |
| compute_derived_variablesDS . . . . .    | 7         |
| detect_type_mismatchesDS . . . . .       | 8         |
| detect_zero_anomaliesDS . . . . .        | 8         |
| drop_rows_missingDS . . . . .            | 9         |
| dummy_encode_ordinalDS . . . . .         | 10        |
| exploratory_analysisDS . . . . .         | 11        |
| fill_missing_dataDS . . . . .            | 12        |
| force_convert_typesDS . . . . .          | 13        |
| global_imputationDS . . . . .            | 13        |
| harmonization_diagnosisDS . . . . .      | 14        |
| harmonization_summaryDS . . . . .        | 15        |
| longitudinal_imputationDS . . . . .      | 16        |
| post_imputation_cleanupDS . . . . .      | 17        |
| remove_columnsDS . . . . .               | 18        |
| <b>Index</b>                             | <b>19</b> |

---

|           |                                       |
|-----------|---------------------------------------|
| cast_NADS | <i>Cast "NA" strings into type NA</i> |
|-----------|---------------------------------------|

---

### Description

Converts "NA" strings of a data frame to proper NA values. Follows the package's disclosure-safe pattern: the transformed data frame is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and is NEVER part of this function's own return value – so even if this function were ever invoked as an aggregate call by mistake, no row-level data crosses the wire. Only a small non-disclosive summary (whether any replacement occurred) is returned.

### Usage

```
cast_NADS(df, newobj = "cast_NA_result")
```

### Arguments

|        |                                                                                                                            |
|--------|----------------------------------------------------------------------------------------------------------------------------|
| df     | A data frame containing values to be converted.                                                                            |
| newobj | Name under which the converted data frame is stored on the server (in the calling environment, i.e. the analytic session). |

**Value**

A list with `newobj` (the name just `base::assign`) and `changed` (logical, whether any "NA" string was replaced).

---

|                  |                                         |
|------------------|-----------------------------------------|
| cast_to_factorDS | <i>Cast specified columns to factor</i> |
|------------------|-----------------------------------------|

---

**Description**

Converts one or more columns of a data frame to factor. Disclosure-safe: the modified data frame is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value.

**Usage**

```
cast_to_factorDS(df, columns, newobj = "cast_to_factor_result")
```

**Arguments**

|                      |                                                                                  |
|----------------------|----------------------------------------------------------------------------------|
| <code>df</code>      | A data frame.                                                                    |
| <code>columns</code> | A character vector (or "\$"-joined string) of column names to convert to factor. |
| <code>newobj</code>  | Name under which the modified data frame is stored.                              |

**Value**

A list with `newobj` and `columns_cast` (the subset of requested columns that were actually present and converted).

---

|                     |                                                |
|---------------------|------------------------------------------------|
| check_categoricalDS | <i>Check validity of categorical variables</i> |
|---------------------|------------------------------------------------|

---

**Description**

Verify that selected categorical variables in a data frame contain only predefined valid values.

**Usage**

```
check_categoricalDS(df)
```

**Arguments**

|                 |                                                                  |
|-----------------|------------------------------------------------------------------|
| <code>df</code> | A data frame containing the categorical variables to be checked. |
|-----------------|------------------------------------------------------------------|

**Value**

A character vector with the names of columns containing invalid values. Returns an empty character vector if all checked variables are valid.

---

check\_missing\_dataDS     *Compute percentage of missing values per column*

---

**Description**

Computes the proportion of missing values (NA) for each column of a data frame. The function returns a named numeric vector containing the percentage of missing values for every variable.

**Usage**

```
check_missing_dataDS(df)
```

**Arguments**

df                      A data frame for which the percentage of missing values per column is to be calculated.

**Value**

A named numeric vector where each element represents the percentage of missing values in the corresponding column of df.

---

check\_numericDS             *Check validity of numeric variables*

---

**Description**

Verify that selected numeric variables in a data frame fall within predefined ranges and satisfy integer constraints where required. Also checks the format of the patient identifier (pat\_ID), if present.

**Usage**

```
check_numericDS(df)
```

**Arguments**

df                      A data frame containing the numeric variables to be checked.

**Value**

A character vector with the names of columns containing invalid values. Returns an empty character vector if all checked variables are valid.

---

check\_patient\_visitDS    *Check patient-visit combinations for duplicates*

---

### Description

Groups by patient identifier and visit column and checks for duplicate (patient, visit) combinations. Disclosure-safe: unlike an earlier version of this function, the full per-patient-visit table (which contained actual patient identifiers) is never returned to the client – only aggregate counts are.

### Usage

```
check_patient_visitDS(
  df,
  pat_id_col = "pat_ID",
  visit_col = "Visit"
)
```

### Arguments

|            |                                                                                 |
|------------|---------------------------------------------------------------------------------|
| df         | A data frame containing at least the columns named by pat_id_col and visit_col. |
| pat_id_col | Character. Name of the patient identifier column. Default "pat_ID".             |
| visit_col  | Character. Name of the visit column. Default "Visit_months_from_diagnosis".     |

### Value

A list:

- n\_unique\_patients, n\_unique\_pairs: counts.
- n\_duplicate\_pairs: number of (patient, visit) combinations occurring more than once.
- max\_occurrences: the largest count observed for any single (patient, visit) pair (1 if no duplicates).

No patient identifier or row-level value is included.

---

check\_residual\_naDS    *Count remaining NAs per column, without dropping anything*

---

### Description

Lets the analyst inspect residual missingness after imputation before deciding whether to call post\_imputation\_cleanup. Never creates a server-side object, so this one is already a plain, disclosure-safe aggregate function – only per-column counts are returned, no row-level data.

### Usage

```
check_residual_naDS(df)
```

### Arguments

|    |               |
|----|---------------|
| df | A data frame. |
|----|---------------|

**Value**

A named numeric vector of remaining NA counts per column (only columns with at least one remaining NA are included).

---

|                   |                                                                               |
|-------------------|-------------------------------------------------------------------------------|
| check_variablesDS | <i>Check that required and optional variables are present in a data frame</i> |
|-------------------|-------------------------------------------------------------------------------|

---

**Description**

Verify that a data frame contains every variable in a required set, report which optional variables (checked for type/range only if present – see `check_numericDS()` / `check_categoricalDS()`) are actually present, and flag anything else in the data frame as unexpected ("extra"). Backward compatible with the original single-list behaviour: if `optional_string` is empty, this reduces to the original "everything not required is extra" check.

**Usage**

```
check_variablesDS(df, variables_string, optional_string = "")
```

**Arguments**

`df` A data frame to be checked.

`variables_string` A single character string specifying the required variable names separated by "\$".

`optional_string` A single character string specifying optional variable names separated by "\$". Defaults to "" (no optional variables declared, i.e. legacy behaviour).

**Value**

A named list with three elements:

- `missing`: required variables not found in the data frame.
- `optional_present`: declared optional variables that ARE present in the data frame (informational only, never a failure).
- `extra`: variables found in the data frame that are neither required nor declared optional.

---

clinical\_variable\_dictionaryDS

*Canonical clinical data dictionary for the harmonization module*


---

### Description

Single source of truth for which variables the RA harmonization protocol expects, split into required (must be present in every site's table) and optional (checked for valid range/type only if present – a site is never flagged for omitting them). This is what `ds.check_variables()` defaults to when the analyst does not supply an explicit variable list, and what `check_numericDS()` / `check_categoricalDS()` validate against.

### Usage

```
clinical_variable_dictionaryDS()
```

### Value

A named list with two character vectors: required and optional.

---

compute\_derived\_variablesDS

*Compute derived clinical variables*


---

### Description

Derives two composite variables when their source columns are available, and leaves the data frame otherwise untouched – some sites may not collect ultrasound data at all, and this should not block the rest of the harmonization pipeline.

- $USPDGS = USPD + USGS$ , computed only if both USPD and USGS are present.
- $DAS2C = \sqrt{SJC28} + 0.6 * \log_{10}(CRP)$ , computed only if both SJC28 and CRP are present.

Row-wise: if either input is NA for a given row, the derived value for that row is NA too. Disclosure-safe: the result is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value.

### Usage

```
compute_derived_variablesDS(df, newobj = "compute_derived_variables_result")
```

### Arguments

|        |                                                      |
|--------|------------------------------------------------------|
| df     | A data frame.                                        |
| newobj | Name under which the resulting data frame is stored. |

### Value

A list with `newobj` and `computed` (character vector of which derived columns were actually added – column NAMES only).

---

detect\_type\_mismatchesDS

*Detect columns whose stored type doesn't match the expected type*


---

### Description

Dry-run companion to force\_convert\_typesDS – reports which columns WOULD change type if conversion were forced, without actually modifying anything. Never touches or returns any data value, only column names, so this one is safe as a plain aggregate return already (no server-side object is created).

### Usage

```
detect_type_mismatchesDS(df, numeric_cols = NULL, categorical_cols = NULL)
```

### Arguments

|                  |                                                                              |
|------------------|------------------------------------------------------------------------------|
| df               | A data frame.                                                                |
| numeric_cols     | Character vector (or "\$"-joined string) of columns expected to be numeric.  |
| categorical_cols | Character vector (or "\$"-joined string) of columns expected to be a factor. |

### Value

A list with numeric\_would\_change and categorical\_would\_change.

---

detect\_zero\_anomaliesDS

*Detect suspicious zero-inflation in numeric columns*


---

### Description

A forced or careless numeric cast can silently turn non-numeric placeholders into zeros instead of NA – e.g. a factor coerced with as.numeric(x) instead of as.numeric(as.character(x)) (returns internal factor level codes, not the labelled values), or an upstream ETL step that filled blanks with 0 before the column ever reached this pipeline. This failure mode is invisible to check\_missing\_dataDS (the cell isn't NA, it's a real-looking 0) and easy to miss visually in a clinical variable where a \*few\* genuine zeros are plausible (e.g. CRP, TJC28), so a purely eyeballed check tends to under-detect it.

Two independent, complementary flags are computed per numeric column:

- **Proportion flag:** the share of non-missing values that are exactly 0 exceeds zero\_prop\_threshold. Simple, but can miss a genuinely zero-inflated variable's true signal vs. a cast artefact, and can also flag a variable that is legitimately often zero (e.g. TJC28 in a well-controlled cohort).
- **Spike flag:** zero-count is compared to the AVERAGE count across equal-width bins spanning the non-zero values – i.e. "does 0 stick out as a spike relative to the local density of the rest of the distribution", which is what an artefact typically looks like (a hard wall at exactly 0 that a smooth clinical distribution wouldn't produce) as opposed to a distribution that is just generally low-valued. Flagged when  $n\_zero / \text{mean}(\text{bin\_counts}) \geq \text{spike\_ratio\_threshold}$ .

A column is reported as anomalous if EITHER flag fires. Neither flag is proof of an error – both are heuristics meant to prompt a human look, not an automatic correction; this function never modifies data.

### Usage

```
detect_zero_anomaliesDS(
  df,
  cols = NULL,
  zero_prop_threshold = 0.3,
  spike_ratio_threshold = 3,
  num_bins = 10,
  nfilter = 5
)
```

### Arguments

|                       |                                                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| df                    | A data frame.                                                                                                                      |
| cols                  | Character vector (or "\$"-joined string) restricting the check to specific columns.<br>NULL (default): check every numeric column. |
| zero_prop_threshold   | Proportion of exact zeros (of non-missing values) above which the proportion flag fires. Default 0.3.                              |
| spike_ratio_threshold | Ratio of zero-count to average non-zero bin count above which the spike flag fires. Default 3.                                     |
| num_bins              | Number of equal-width bins used for the spike check across the non-zero values. Default 10.                                        |
| nfilter               | Minimum number of non-missing values required before a column is evaluated at all (disclosure/stability floor).                    |

### Value

A named list, one entry per checked numeric column with at least nfilter non-missing values, each containing n\_nonmissing, n\_zero, prop\_zero, spike\_ratio, prop\_flag, spike\_flag, flagged (either flag fired).

---

|                     |                                                           |
|---------------------|-----------------------------------------------------------|
| drop_rows_missingDS | <i>Drop rows with missing values in specified columns</i> |
|---------------------|-----------------------------------------------------------|

---

### Description

Companion to remove\_columnsDS for the missing-data workflow: instead of dropping a variable that exceeds the missingness threshold, this drops the ROWS that are missing on that variable, keeping the variable itself. Disclosure-safe: the resulting data frame is stored server-side via base::assign(newobj, ..., envir = parent.frame()) and never appears in this function's own return value – only row counts (aggregate, non-disclosive) are returned.

### Usage

```
drop_rows_missingDS(df, col_names, newobj = "drop_rows_missing_result")
```

**Arguments**

|           |                                                                                      |
|-----------|--------------------------------------------------------------------------------------|
| df        | A data frame.                                                                        |
| col_names | A character vector (or "\$"-joined string) of column names to check for missingness. |
| newobj    | Name under which the resulting data frame is stored.                                 |

**Value**

A list with newobj, n\_before, n\_after, n\_removed.

---

dummy\_encode\_ordinalDS

*Dummy-encode ordinal/nominal-coded variables*

---

**Description**

For each named column, coerces it to integer (keeping the ORIGINAL column, unlike `cast_to_factorDS` which replaces it) and adds one integer 0/1 dummy column per observed level, named "<column>\_<level>". Rows with NA in the source column get NA in every dummy for that column.

IP\_score and EP\_score are explicitly excluded even if passed in columns – these are patient-annotation severity scores (0-4), not treatment-class categories.

Disclosure-safe: the result is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value – only column NAMES are returned.

**Usage**

```
dummy_encode_ordinalDS(df, columns, newobj = "dummy_encode_ordinal_result")
```

**Arguments**

|         |                                                                        |
|---------|------------------------------------------------------------------------|
| df      | A data frame.                                                          |
| columns | A character vector (or "\$"-joined string) of columns to dummy-encode. |
| newobj  | Name under which the resulting data frame is stored.                   |

**Value**

A list with newobj, added (new dummy column names), and skipped\_excluded (requested columns skipped because they're on the excluded list).

---

exploratory\_analysisDS

*Non-disclosive exploratory analysis*


---

## Description

Computes descriptive statistics and plotting-ready summaries for exploratory data analysis, without ever returning a row-level value. Companion to `harmonization_summaryDS` (which covers missingness- oriented reporting); this one is for understanding the SHAPE of the (typically post-cleaning) data: distributions, category frequencies, correlations between numeric variables, and simple group-wise trends (e.g. a variable's mean at each visit number).

Every returned number is either a count/proportion, a summary statistic with an `nfilter` floor, or a bin/cell count with an `nfilter` floor – the same disclosure-control convention used throughout this package. Numeric distributions are summarized as BINNED HISTOGRAM COUNTS (shared bin edges built from the column's own min/max), never as raw values, mirroring the federated-histogram approach used elsewhere in this codebase for exactly the same reason: a bin count summarizes an interval, not one identifiable value.

## Usage

```
exploratory_analysisDS(
  df,
  numeric_cols = NULL,
  categorical_cols = NULL,
  group_col = NULL,
  num_bins = 20,
  nfilter = 5
)
```

## Arguments

|                                             |                                                                                                                                                                   |
|---------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>                             | A data frame.                                                                                                                                                     |
| <code>numeric_cols, categorical_cols</code> | Character vectors (or "\$"-joined strings) restricting which columns to summarize. NULL (default): every numeric / every non-numeric column respectively.         |
| <code>group_col</code>                      | Optional column (e.g. "Visit") to compute simple group-wise means of each numeric column against – useful for a longitudinal trend plot. NULL (default): skipped. |
| <code>num_bins</code>                       | Number of bins for the numeric histograms. Default 20.                                                                                                            |
| <code>nfilter</code>                        | Minimum count required before any statistic/cell/bin is released.                                                                                                 |

## Value

A list:

- `n`: row count at this site.
- `numeric_summary`: as in `harmonization_summaryDS` (`n`, mean, sd, min, q25, median, q75, max per numeric column).
- `numeric_histograms`: named list, one entry per numeric column with breaks (bin edges) and counts (per-bin counts, NA where a bin's count is below `nfilter`).

- `categorical_summary`: named list, one entry per categorical column, itself a named list of category -> count (counts below `nfilter` replaced with NA).
- `correlation`: `list(cols=, matrix=)`, a Pearson correlation matrix over numeric columns with pairwise-complete observations, cells set to NA where the pairwise complete count is below `nfilter`. NULL if fewer than 2 numeric columns are available.
- `group_trend`: `list(levels=, means=)`, per-numeric- column mean at each level of `group_col` (NA where the group's count is below `nfilter`). NULL if `group_col` wasn't supplied or isn't present.

---

|                                  |                                                       |
|----------------------------------|-------------------------------------------------------|
| <code>fill_missing_dataDS</code> | <i>Fill missing values by patient and visit order</i> |
|----------------------------------|-------------------------------------------------------|

---

## Description

For each patient, sorts visits by time from diagnosis and propagates non-missing values of a specified column forward chronologically within the same patient. Disclosure-safe: the filled data frame is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value.

## Usage

```
fill_missing_dataDS(
  df,
  pat_id_col,
  visit_col,
  value_col,
  newobj = "fill_missing_data_result"
)
```

## Arguments

|                         |                                                                                                                       |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------|
| <code>df</code>         | A data frame containing at least the patient identifier column, the visit time column, and the variable to be filled. |
| <code>pat_id_col</code> | A character string specifying the name of the patient identifier column.                                              |
| <code>visit_col</code>  | A character string specifying the name of the visit time column.                                                      |
| <code>value_col</code>  | A character string specifying the name of the column whose missing values should be filled.                           |
| <code>newobj</code>     | Name under which the filled data frame is stored.                                                                     |

## Value

A list with `newobj`, `n_rows`, and `n_filled` (count of previously-missing `value_col` cells that received a carried-forward value – an aggregate count, non-disclosive).

---

force\_convert\_typesDS *Force type conversion on selected columns*

---

### Description

Coerces the named columns to numeric and/or factor. Disclosure-safe: the result is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value – only column names and, for the numeric columns just converted, non-disclosive zero-anomaly diagnostics (see `detect_zero_anomaliesDS`) are returned.

### Usage

```
force_convert_typesDS(
  df,
  numeric_cols = NULL,
  categorical_cols = NULL,
  newobj = "force_convert_types_result",
  zero_prop_threshold = 0.3,
  spike_ratio_threshold = 3
)
```

### Arguments

|                                                         |                                                                                                                 |
|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>df</code>                                         | A data frame.                                                                                                   |
| <code>numeric_cols</code>                               | Character vector (or "\$"-joined string) of columns to force to numeric.                                        |
| <code>categorical_cols</code>                           | Character vector (or "\$"-joined string) of columns to force to factor.                                         |
| <code>newobj</code>                                     | Name under which the resulting data frame is stored.                                                            |
| <code>zero_prop_threshold, spike_ratio_threshold</code> | Passed through to <code>detect_zero_anomaliesDS</code> for the automatic post-conversion check on numeric_cols. |

### Value

A list with `newobj`, `numeric_converted`, `categorical_converted`, and `zero_anomalies` (diagnostics for the newly-converted numeric columns; empty list if none were converted).

---

global\_imputationDS *Global imputation of missing data using MICE*

---

### Description

Performs a global (pooled, not patient-grouped) imputation of missing values using Multiple Imputation by Chained Equations (MICE): identifies continuous vs. categorical variables, `base::assign`s appropriate methods (`pmm`, `logreg`, `polyreg`), and returns a single averaged/modal-consensus imputed data frame. Disclosure-safe: the imputed data frame is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value – only a small summary (row/column counts and the per-column imputation METHOD names, e.g. "pmm"/"logreg" – never any data value) is returned.

**Usage**

```
global_imputationDS(
  df,
  id_col,
  m = 5,
  maxit = 30,
  newobj = "global_imputation_result"
)
```

**Arguments**

|                     |                                                                                                            |
|---------------------|------------------------------------------------------------------------------------------------------------|
| <code>df</code>     | A data frame containing the variables to be imputed.                                                       |
| <code>id_col</code> | A string specifying the name of the identifier column. Excluded from imputation and reattached afterwards. |
| <code>m</code>      | Number of imputations to generate (passed to mice).                                                        |
| <code>maxit</code>  | Maximum number of MICE iterations per imputation chain.                                                    |
| <code>newobj</code> | Name under which the imputed data frame is stored.                                                         |

**Value**

A list with `newobj`, `n_rows`, `n_cols`, and `method_by_column` (named character vector of MICE methods used per column – method names only, never data).

---

harmonization\_diagnosisDS

*Consolidated, non-disclosive harmonization diagnosis*

---

**Description**

Bundles every existing check in this package (`check_variablesDS`, `check_numericDS`, `check_categoricalDS`, `check_missing_dataDS`, `check_patient_visitDS`, `detect_zero_anomaliesDS`) plus a check for "NA"-string contamination into a single aggregate call, so a full site diagnosis takes one round trip instead of six. Used by `ds.harmonization_orchestrator()` for both the "before" and "after" snapshots, but also usable standalone.

**Usage**

```
harmonization_diagnosisDS(
  df,
  required_vars,
  optional_vars,
  pat_id_col = "pat_ID",
  visit_col = "Visit",
  nfilter = 5,
  zero_prop_threshold = 0.3,
  spike_ratio_threshold = 3
)
```

**Arguments**

df                      A data frame.

required\_vars, optional\_vars                      Character vectors (or "\$"-joined strings) – see check\_variablesDS.

pat\_id\_col, visit\_col                      See check\_patient\_visitDS. Duplicate checking is skipped (returns NA) if either column is absent.

nfilter                      Disclosure/stability floor, passed through to every sub-check that has one.

zero\_prop\_threshold, spike\_ratio\_threshold                      See detect\_zero\_anomaliesDS.

**Value**

A list:

- n: row count at this site.
- missing\_required, optional\_present, extra\_vars: from check\_variablesDS.
- na\_string\_present: logical, whether any character column contains the literal string "NA" (a sign cast\_NADS hasn't been run yet).
- invalid\_numeric, invalid\_categorical: column names failing range/value checks.
- missing\_pct: named numeric vector, % missing per column.
- duplicate\_patient\_visit\_pairs: count of patient-visit pairs occurring more than once, or NA if the id/visit columns aren't both present.
- zero\_anomalies: output of detect\_zero\_anomaliesDS.

---

harmonization\_summaryDS

*Non-disclosive harmonization summary*

---

**Description**

Produces exactly the aggregate statistics needed to build a harmonization report/figures client-side, and nothing else – no row-level value ever leaves the server. Every returned number is either a count, a percentage, or a summary statistic (mean/sd/quantile) with a minimum-count floor enforced first via nfilter.

**Usage**

```
harmonization_summaryDS(df, nfilter = 5)
```

**Arguments**

df                      A data frame.

nfilter                      Minimum number of non-missing observations required before a numeric summary or a categorical cell count is released. Categorical cells below this count are reported as NA (suppressed), same convention as other DataSHIELD packages (e.g. ds.table).

**Value**

A list:

- `n`: total row count at this site.
- `missing_pct`: named numeric vector,
- `numeric_summary`: named list, one entry per numeric column with `n`, mean, sd, min, q25, median, q75, max (entry omitted entirely if non-missing `n < nfilter`).
- `categorical_summary`: named list, one entry per non-numeric column, itself a named list of category -> count (counts below `nfilter` replaced with NA).

---

longitudinal\_imputationDS

*Longitudinal (patient-grouped) MICE imputation*

---

**Description**

The preferred imputation method for this harmonization module: for each patient, orders their visits chronologically and runs predictive mean matching (`mice`, `method = "pmm"`) WITHIN that patient's own visit history, then averages across the `m` imputed draws per numeric variable. This mirrors a clinically sensible assumption – that a patient's own trajectory is the best predictor of their missing values – rather than pooling across unrelated patients.

This is an adapted, corrected version of the originally supplied `impute_missing_data_mice()` sketch:

- the join key at the end used the literal column `Visit` even when the function was called with a different `visit_col` (e.g. `Protocol_visit`) – fixed to always use the actual `visit_col` argument.
- `quickpred(..., exclude = c("sam_ID", "Protocol_visit"))` hardcoded two specific column names regardless of what the caller's data actually contained – replaced with an `exclude_cols` argument the caller controls.
- a patient with a single visit (or fewer rows than `mice` can meaningfully model) would previously error out the whole call – such patients are now passed through unimputed with a per-site warning rather than crashing the pipeline.
- a disclosure floor (`nfilter`) is enforced on site size, consistent with every other function in this package.

**Usage**

```
longitudinal_imputationDS(
  df,
  pat_id_col,
  visit_col,
  exclude_cols = NULL,
  m = 5,
  maxit = 30,
  nfilter = 5,
  newobj = "longitudinal_imputation_result"
)
```

**Arguments**

|                           |                                                                                                                                                                                                            |
|---------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>df</code>           | A data frame containing at least <code>pat_id_col</code> and <code>visit_col</code> .                                                                                                                      |
| <code>pat_id_col</code>   | Character. Name of the patient identifier column.                                                                                                                                                          |
| <code>visit_col</code>    | Character. Name of the visit-ordering column (e.g. <code>Visit</code> or <code>Visit_months_from_diagnosis</code> ).                                                                                       |
| <code>exclude_cols</code> | Character vector (or "\$"-joined string), columns to exclude from <code>mice::quickpred</code> 's predictor matrix (e.g. free-text identifiers). <code>pat_id_col</code> is always excluded automatically. |
| <code>m</code>            | Number of imputations per patient (passed to <code>mice</code> ).                                                                                                                                          |
| <code>maxit</code>        | Maximum MICE iterations per chain.                                                                                                                                                                         |
| <code>nfilter</code>      | Minimum site row count required to proceed at all.                                                                                                                                                         |
| <code>newobj</code>       | Name under which the resulting data frame is stored.                                                                                                                                                       |

**Value**

Disclosure-safe: the imputed data frame (one row per patient, visit; numeric columns imputed and averaged across the `m` draws; non-numeric columns taken from the modal imputed value) is stored server-side via `base::assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value. Returns a list with `newobj`, `n_rows`, `n_patients`, and `n_patients_unimputed` (patients with <2 visits, left as-is).

---

`post_imputation_cleanupDS`

*Drop rows that are still incomplete after imputation*

---

**Description**

Imputation can legitimately leave some rows with residual NAs. This function lets the analyst explicitly drop them after seeing how many rows are affected (via `check_residual_naDS`); it never happens implicitly inside the imputation step itself. Disclosure-safe: the result is stored server-side via `base::(newobj, ..., envir = parent.frame())` and never appears in this function's own return value – only row counts are.

**Usage**

```
post_imputation_cleanupDS(df, newobj = "post_imputation_cleanup_result")
```

**Arguments**

|                     |                                                            |
|---------------------|------------------------------------------------------------|
| <code>df</code>     | A data frame (typically the output of an imputation step). |
| <code>newobj</code> | Name under which the resulting data frame is stored.       |

**Value**

A list with `newobj`, `n_before`, `n_after`, `n_removed`.

|                  |                                 |
|------------------|---------------------------------|
| remove_columnsDS | <i>Remove specified columns</i> |
|------------------|---------------------------------|

---

**Description**

Drops one or more named columns from a data frame. Disclosure-safe: the resulting data frame is stored server-side via `assign(newobj, ..., envir = parent.frame())` and never appears in this function's own return value.

**Usage**

```
remove_columnsDS(df, col_names, newobj = "remove_columns_result")
```

**Arguments**

|                        |                                                                       |
|------------------------|-----------------------------------------------------------------------|
| <code>df</code>        | A data frame.                                                         |
| <code>col_names</code> | A character vector (or "\$"-joined string) of column names to remove. |
| <code>newobj</code>    | Name under which the resulting data frame is stored.                  |

**Value**

A list with `newobj`, `columns_removed` (subset of the requested columns that were actually present), `n_cols_before`, and `n_cols_after`.

# Index

cast\_NADS, [2](#)  
cast\_to\_factorDS, [3](#)  
check\_categoricalDS, [3](#)  
check\_missing\_dataDS, [4](#)  
check\_numericDS, [4](#)  
check\_patient\_visitDS, [5](#)  
check\_residual\_naDS, [5](#)  
check\_variablesDS, [6](#)  
clinical\_variable\_dictionaryDS, [7](#)  
compute\_derived\_variablesDS, [7](#)  
  
detect\_type\_mismatchesDS, [8](#)  
detect\_zero\_anomaliesDS, [8](#)  
drop\_rows\_missingDS, [9](#)  
dummy\_encode\_ordinalDS, [10](#)  
  
exploratory\_analysisDS, [11](#)  
  
fill\_missing\_dataDS, [12](#)  
force\_convert\_typesDS, [13](#)  
  
global\_imputationDS, [13](#)  
  
harmonization\_diagnosisDS, [14](#)  
harmonization\_summaryDS, [15](#)  
  
longitudinal\_imputationDS, [16](#)  
  
post\_imputation\_cleanupDS, [17](#)  
  
remove\_columnsDS, [18](#)